

Clase 245 — Gestión de vulnerabilidades a escala

Parte: **11 — DevSecOps y seguridad del SDLC** · Fuente: *Securing DevOps* (Julien Vehent) y NIST SP 800-40r4 (Guide to Enterprise Patch Management) 🕒 Duración estimada: **110 min** · Nivel: **Avanzado**

🎯 Objetivo

Pasar de "escanear y encontrar vulnerabilidades" a **operar un programa de gestión de vulnerabilidades**: consolidar hallazgos de muchas herramientas, deduplicarlos, priorizarlos por riesgo real, asignarlos con SLA, seguir su remediación y medir el programa con métricas. Cuando tienes cientos de repos y miles de hallazgos, sin un proceso te ahogas; con proceso, reduces el riesgo de forma medible.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Diseñar** el ciclo de vida de una vulnerabilidad (descubrir → triage → priorizar → remediar → verificar).
- 2. **Priorizar** con un modelo de riesgo (CVSS + EPSS + KEV + exposición + criticidad del activo).
- 3. **Definir** SLAs de remediación por severidad y medir su cumplimiento.
- 4. **Consolidar** hallazgos de múltiples fuentes evitando duplicados y fatiga.
- 5. **Reportar** el estado con métricas útiles para dirección y equipos.

📖 Temas

#	Tema	Por qué importa
1	Ciclo de vida de la vulnerabilidad	El proceso, no solo el escaneo
2	Consolidación y deduplicación	Muchas herramientas, un backlog
3	Priorización basada en riesgo	No todo lo crítico es urgente
4	SLAs por severidad	Compromisos medibles de remediación
5	VEX y excepciones	Documentar lo no explotable
6	Métricas del programa	MTTR, backlog, tasa de recurrencia
7	Herramientas de agregación	DefectDojo y similares

📖 Definiciones y características

- **Gestión de vulnerabilidades**: proceso continuo de identificar, evaluar, tratar y reportar vulnerabilidades. *Característica*: es un ciclo, no un evento puntual.
- **Triage**: clasificar un hallazgo (real/falso, severidad, dueño). *Característica*: filtra ruido antes de asignar trabajo.
- **Priorización basada en riesgo (RBVM)**: ordenar por riesgo real, no solo por CVSS. *Característica*: combina severidad, explotabilidad, exposición y valor del activo.

- **SLA de remediación:** plazo máximo para corregir según severidad. *Característica:* hace el compromiso medible y auditable.
- **VEX:** Vulnerability Exploitability eXchange, documento que declara si un CVE es explotable en tu producto. *Característica:* reduce ruido justificando "not affected".
- **MTTR:** tiempo medio de remediación. *Característica:* métrica clave de la salud del programa.

Herramientas y preparación

- **DefectDojo** (open source) — plataforma de gestión y consolidación de hallazgos.
- **Trivy / Semgrep / ZAP** — fuentes de hallazgos (clases anteriores).
- **CISA KEV** y **EPSS** — señales de priorización.
- Hoja de cálculo o issue tracker (Jira/GitHub Issues) para el flujo si no usas DefectDojo.

```
# DefectDojo de práctica con Docker Compose:
git clone https://github.com/DefectDojo/django-DefectDojo
cd django-DefectDojo && docker compose up -d
```

Laboratorio guiado

Ejercicio de proceso (defensivo, sobre hallazgos propios):

1. **Recolecta hallazgos.** Exporta resultados de Trivy (SCA/imagen), Semgrep (SAST) y ZAP (DAST) de tus prácticas en formato JSON.
2. **Consolida en DefectDojo.** Crea un producto y engagement, importa los tres informes. Observa cómo deduplica hallazgos repetidos entre escaneos.
3. **Enriquece con señales de riesgo.** Para los top 20 hallazgos, añade EPSS y marca los que están en CISA KEV.
4. **Prioriza.** Ordena por un score compuesto: severidad × exposición (¿internet-facing?) × criticidad del activo, elevando los KEV/EPSS alto. Documenta el criterio.
5. **Asigna SLAs.** Define plazos, p. ej.: Crítica explotable 7 días, Alta 30, Media 90, Baja 180. Asigna dueño a cada hallazgo top.
6. **Documenta excepciones con VEX.** Para un CVE en una función que no usas, redacta un statement "not affected" con justificación.
7. **Genera métricas.** Calcula backlog por severidad, MTTR del último periodo y tasa de recurrencia (hallazgos que reaparecen). Prepara un mini-dashboard.

Nota ética: la gestión de vulnerabilidades es puramente defensiva. Los datos de hallazgos son sensibles; trátalos con control de acceso y no los publiques.

Ejercicios

1. Dibuja el ciclo de vida de una vulnerabilidad en tu organización.
2. Consolida hallazgos de dos herramientas y cuenta los duplicados eliminados.
3. Prioriza 10 hallazgos con un modelo que combine CVSS, EPSS y KEV.
4. Define una tabla de SLAs por severidad y justifica los plazos.
5. Redacta un statement VEX "not affected" para un CVE concreto.
6. Calcula el MTTR de un conjunto de hallazgos con fechas de apertura/cierre.

Reto verificable

Opera un mini-programa de gestión de vulnerabilidades sobre hallazgos reales de tus prácticas.

Criterio de aceptación: (a) los hallazgos de al menos tres herramientas están consolidados y deduplicados; (b) existe un modelo de priorización documentado que va más allá de CVSS (incluye EPSS/KEV/exposición); (c) hay SLAs por severidad y cada hallazgo top tiene dueño y plazo; (d) al menos una excepción está documentada como VEX; y (e) se reporta MTTR y backlog por severidad.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Backlog de 10.000 hallazgos sin tocar	Priorizas por CVSS crudo. Filtra por explotabilidad y exposición; ataca lo alcanzable primero.
El mismo bug aparece 30 veces	No hay deduplicación. Usa una plataforma (DefectDojo) que fusione hallazgos equivalentes.
Los equipos ignoran los tickets	Sin SLA ni dueño claro. Asigna responsable y plazo, y escala incumplimientos.
Se remedia pero reaparece	No se corrigió la causa raíz (dependencia base). Mide tasa de recurrencia y ataca el origen.
Métricas que nadie usa	Reportas número de CVE sin contexto. Mide MTTR, backlog por riesgo y tendencia.

Preguntas frecuentes

? ¿Por qué no arreglar simplemente todo lo "crítico"? Porque "crítico" en CVSS no significa "explotable en tu contexto". Un crítico sin exploit y sin exposición puede esperar frente a un alto que está en KEV y es internet-facing.

? ¿Qué SLAs son razonables? Depende del riesgo y del sector, pero un patrón común: crítico explotable en días, alto en semanas, medio en meses. Lo importante es que sean explícitos y medibles.

? ¿Necesito DefectDojo o basta con Jira? Para pocos repos, un tracker con buena disciplina sirve. A escala, una plataforma que importe múltiples formatos, deduplique y calcule métricas ahorra muchísimo esfuerzo.

? ¿Qué es y para qué sirve VEX? Es una forma estandarizada de decir "este CVE está presente pero no somos explotables porque...". Reduce el ruido y evita remediar lo que no aplica, con trazabilidad.

Referencias

- NIST SP 800-40r4 Patch Management — <https://csrc.nist.gov/pubs/sp/800/40/r4/final>
- DefectDojo — <https://www.defectdojo.org/>
- CISA KEV — <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
- EPSS — <https://www.first.org/epss/>
- CycloneDX VEX — <https://cyclonedx.org/capabilities/vex/>

Siguiente clase

Clase 246 - Supply chain security: SBOM y SLSA